EmoSense — Backend Documentation

FastAPI + ONNX Runtime service: app.py, requirements and Dockerfile explained

Contents of this document

- 02-backend/README.md

Generated from the Markdown files in the docs folder (the Markdown files remain the source of truth).

2. The backend (FastAPI + ONNX Runtime)

Source: `backend/app.py`, `backend/Dockerfile`, `backend/requirements.txt`, `render.yaml`

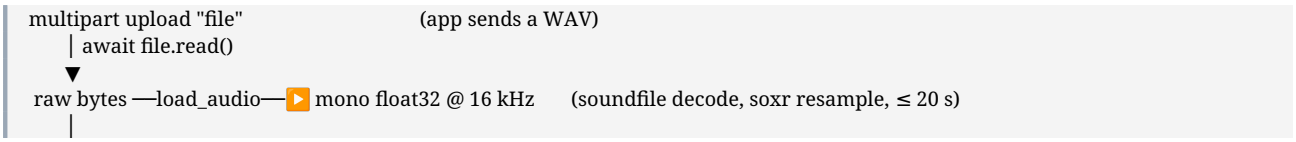
Purpose

A small web service that receives **an audio file**, runs the emotion model, and answers with **JSON**: the emotion, the confidence and all probabilities. The Android app (or Postman, or a browser) talks to it over HTTPS.

Why this component

Piece	What	Why chosen	Rejected alternative
FastAPI	Python web framework	Very little code; automatic request validation; built-in `/docs` page for testing	Flask (no async/validation helpers), Django (far too heavy)
ONNX Runtime	Runs `model.onnx` on CPU	~20 MB library, no PyTorch needed	PyTorch (≈ 700 MB image, does not fit a free server)
soundfile + **soxr**	Decode audio / resample to 16 kHz	Tiny, fast, no numba	`librosa` (pulls numba/scipy: slow start, more RAM)
NumPy `trim_silence`	Same silence trimming as the notebook, without librosa	Keeps train/serve parity at low RAM	Skipping trimming: predictions would differ from the experiments
Sliding windows (windows)	3 s windows, overlap for short clips	The model was trained on 3 s	Feeding a long clip at once: wrong input size
One window per model call	`batch = 1`	Batching several windows made the free instance run out of memory (HTTP 502 seen in testing, most likely an out-of-memory crash)	Batching (faster but crashed)
`MAX_WINDOWS`, `MAX_SECONDS`	Caps on work per request	A fractional CPU would otherwise time out	Unlimited length
Energy gate `MIN_RMS`	Silent window → "no speech"	Never classify silence	Always returning an emotion
Warm-up call	One dummy inference at start	First real request is not slow	Lazy first call
CORS middleware	Allows any origin	Lets a browser/Postman web client call the API	No CORS: browser calls blocked
Docker	Packaged server	Render builds and runs the same image every time	Installing Python by hand on the server

Diagram: what happens inside `POST /predict`



```
    ▼ windows(y)  → list of (start, 3 s segment)      (≤ 5 windows)
    │ for each window:
    │   quiet?  (RMS < 0.003) ──yes──▶ mark "speech": false, skip
    │   no ──▶ to_fixed(): trim silence, crop/pad to 48 000 samples
    ▼
for each kept window: SESSION.run(...) → 7 logits      (ONNX Runtime, batch = 1)
    ▼
softmax → probabilities per window → average over windows
    ▼
JSON: emotion, confidence, probabilities, windows, speech_detected, latency_ms
```

Line-by-line walkthrough: `backend/app.py`

Header and imports

```
import io, json, os, time
```

`io` wraps bytes as a file-like object (so `soundfile` can read from memory), `json` reads `labels.json`, `os` reads environment variables, `time` measures latency.

```
import numpy as np
import onnxruntime as ort
import soundfile as sf
import soxr
from fastapi import FastAPI, File, HTTPException, UploadFile
from fastapi.middleware.cors import CORSMiddleware
```

NumPy for arrays; ONNX Runtime for the neural network; `sf` decodes audio; `soxr` resamples; FastAPI pieces: the app object, `File`/`UploadFile` for uploads, `HTTPException` to return error codes, and the CORS middleware.

Settings from environment variables

```
MODEL_DIR = os.environ.get("MODEL_DIR", os.path.join(os.path.dirname(__file__), "model"))
MODEL_FILE = os.environ.get("MODEL_FILE", "model.onnx")
```

Where the model lives (`backend/model/` by default) and which file to load. Environment variables let you switch the file (for example to the int8 version) **in the Render dashboard without editing code**.

```
MAX_SECONDS = float(os.environ.get("MAX_SECONDS", "20"))      # reject / truncate very long uploads
MAX_WINDOWS = int(os.environ.get("MAX_WINDOWS", "5"))        # cap work per request on a tiny CPU
MIN_RMS = float(os.environ.get("MIN_RMS", "0.003"))          # energy gate for "no speech"
```

Three safety limits: audio longer than 20 s is cut, at most 5 windows are analysed (about 15 s), and a window whose loudness (RMS) is below 0.003 counts as silence.

```
meta = json.load(open(os.path.join(MODEL_DIR, "labels.json")))
LABELS, SR, N = meta["labels"], int(meta["sample_rate"]), int(meta["window_samples"])
TOP_DB = int(meta.get("top_db", 30))
```

Read the contract written by the notebook: class names in output order, sample rate (16 000), window length (48 000 samples) and the trimming threshold (30 dB). **The server never hard-codes the label order**, so retraining with a different list cannot silently mislabel results.

```
so = ort.SessionOptions()
so.intra_op_num_threads = int(os.environ.get("ORT_THREADS", "2"))
SESSION = ort.InferenceSession(os.path.join(MODEL_DIR, MODEL_FILE), so, providers=["CPUExecutionProvider"])
```

Create the **inference session once at start-up** (loading a model takes seconds; doing it per request would be far too slow). `ORT_THREADS` limits CPU threads (the Dockerfile sets 1 for the free plan); `CPUExecutionProvider` = run on the CPU.

```
try:
    SESSION.run(None, {"waveform": np.zeros((1, N), np.float32)})
except Exception:
    pass
```

Warm-up: run one silent window through the network. The first call initialises memory and kernels; doing it now means the first user does not wait longer. Errors are ignored so a warm-up problem can never stop the server from starting.

```
app = FastAPI(title="Speech Emotion Recognition API", version="1.0")
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])
```

Create the application object (`title` appears on `/docs`) and allow requests from any origin.

`trim_silence` — librosa's trimming without librosa

```
def trim_silence(y: np.ndarray, top_db: float = TOP_DB, frame: int = 2048, hop: int = 512) -> np.ndarray:
    if len(y) < frame:
        return y
```

Very short audio (< 2048 samples = 0.13 s) is returned unchanged.

```
    yp = np.pad(y, (frame // 2, frame // 2))                # centre frames, zero padding
    n = 1 + (len(yp) - frame) // hop
    idx = np.arange(frame)[None, :] + hop * np.arange(n)[:, None]
```

Pad the signal and compute index numbers for **overlapping frames of 2048 samples, 512 apart** — a table with one row per frame.

```
    power = np.mean(yp[idx] ** 2, axis=1)                   # rms**2 per frame
    db = 10 * np.log10(np.maximum(1e-10, power)) - 10 * np.log10(np.maximum(1e-10, power.max()))
```

Energy of each frame (mean of squares), converted to **decibels relative to the loudest frame** (`1e-10` avoids log of zero). So the loudest frame is 0 dB and quieter frames are negative.

```
    nz = np.flatnonzero(db > -top_db)
    if len(nz) == 0:
        return y[:0]
    return y[nz[0] * hop: min(len(y), (nz[-1] + 1) * hop)]
```

Frames louder than −30 dB count as sound. Return the audio from the first to the last such frame — the **trimmed** clip (or an empty array if nothing qualifies). This reproduces `librosa.effects.trim`; during development the two functions returned identical results on 30 test signals.

`to_fixed` — exact copy of the notebook function

```
def to_fixed(y: np.ndarray) -> np.ndarray:
    if len(y) == 0:
        return np.zeros(N, np.float32)
    yt = trim_silence(y)
    if len(yt) < 0.1 * SR:
        yt = y
```

Empty → silent clip; if trimming leaves < 0.1 s keep the original.

```
    if len(yt) >= N:
```

```
    s = (len(yt) - N) // 2
    yt = yt[s:s + N]
  else:
    yt = np.pad(yt, (0, N - len(yt)))
  return yt.astype(np.float32)
```

Centre-crop to 3 s, or zero-pad up to 3 s. (Explained in ../01-ml/02-preprocessing-augmentation.md.)

load_audio

```
def load_audio(raw: bytes) -> np.ndarray:
  y, sr = sf.read(io.BytesIO(raw), dtype="float32", always_2d=True, frames=int(MAX_SECONDS * 48000))
```

Decode the uploaded bytes **in memory** (`io.BytesIO`; nothing is written to disk — important on a read-only/ephemeral host). `always_2d=True` gives a `(samples, channels)` shape even for mono files; `frames=` stops reading after 20 s of audio at up to 48 kHz, protecting memory against huge uploads.

```
  y = y.mean(axis=1)
  if sr != SR:
    y = soxr.resample(y, sr, SR, quality="HQ")
  return y[: int(MAX_SECONDS * SR)].astype(np.float32)
```

Average the channels to **mono**; resample to 16 kHz if needed (the app already sends 16 kHz, so this normally does nothing, but Postman uploads may differ); enforce the length limit; return 32-bit floats.

softmax

```
def softmax(z):
  e = np.exp(z - z.max(axis=-1, keepdims=True))
  return e / e.sum(axis=-1, keepdims=True)
```

Turns the 7 raw scores (logits) into probabilities that sum to 1. Subtracting the maximum first is a standard trick that avoids numerical overflow without changing the result.

windows

```
  if len(y) <= N:
    yield 0, y
    return
```

Up to 3 s: one window.

```
  hop = N // 2 if len(y) <= 2 * N else N
```

Clips up to 6 s use a hop of **1.5 s (50 % overlap)**; longer clips use a hop of 3 s (no overlap) so that a 15-second recording needs only 5 model calls on the slow free CPU.

```
  for s in range(0, len(y), hop):
    seg = y[s:s + N]
    if len(seg) < N // 3 and s > 0:       # ignore a tiny tail
      break
    yield s, seg
    if s + N >= len(y):
      break
```

Walk through the signal; ignore a final fragment shorter than 1 s (too little information); stop after the window that reaches the end. `yield` makes `windows` a generator: it hands out one `(start, segment)` pair at a time.

Routes

```
@app.get("/health")
def health():
    return {"status": "ok", "model": MODEL_FILE, "labels": LABELS}
```

A cheap URL that returns immediately. **Render and UptimeRobot call it** to know the service is alive, and the app calls it to show "Server online".

POST /predict

```
@app.post("/predict")
async def predict(file: UploadFile = File(...)):
    t0 = time.time()
    raw = await file.read()
    if not raw:
        raise HTTPException(400, "empty file")
```

`UploadFile = File(...)` declares a **required multipart field called file** (FastAPI returns HTTP 422 automatically if it is missing). `async`/`await` lets the server accept other requests while the upload is being read. An empty upload gives **400 Bad Request**.

```
    try:
        y = load_audio(raw)
    except Exception as e:
        raise HTTPException(400, f"could not decode audio: {type(e).__name__}") from e
```

A file that is not audio (e.g. a text file) produces a clear 400 error instead of a crash (HTTP 500).

```
    segs, wins = [], []
    for s, seg in list(windows(y))[:MAX_WINDOWS]:
        rms = float(np.sqrt(np.mean(seg ** 2))) if len(seg) else 0.0
        if rms < MIN_RMS:                       # energy gate: skip silent windows
            wins.append({"start_s": round(s / SR, 2), "speech": False})
            continue
        segs.append((len(wins), to_fixed(seg)))
        wins.append({"start_s": round(s / SR, 2), "speech": True})
```

For each window (at most 5): compute loudness (RMS); silent windows are only recorded as `"speech": false`; others are preprocessed with `to_fixed` and queued. `len(wins)` remembers the window's position so results can be attached to the right entry later.

```
    if not segs:
        return {"emotion": None, "confidence": 0.0, "probabilities": {}, "windows": wins, "speech_detected": False,
                "latency_ms": round((time.time() - t0) * 1000, 1)}
```

If **every** window was silent, answer `speech_detected: false` — the app then says "I could not hear any speech".

```
    probs = softmax(np.concatenate([SESSION.run(None, {"waveform": x[None]})[0] for _, x in segs]))   # (n_windows, n_classes)
```

The core line: for each window, `x[None]` adds the batch dimension (shape `(1, 48000)`), `SESSION.run` returns `[logits]`, `[0]` takes the logits array, `np.concatenate` stacks all windows into `(n_windows, 7)` and `softmax` converts to probabilities. **One window per call** keeps memory flat on a 512 MB instance.

```
    for (wi, _), p in zip(segs, probs):
        wins[wi].update(emotion=LABELS[int(p.argmax())], confidence=round(float(p.max()), 4))
    mean_p = probs.mean(axis=0)
    k = int(mean_p.argmax())
```

Attach each window's own best emotion and confidence to its entry (this becomes the timeline in the app), **average the probabilities over windows** (`mean_p`) and pick the overall winner `k`.

```
    return {"emotion": LABELS[k], "confidence": round(float(mean_p[k]), 4),
            "probabilities": {l: round(float(v), 4) for l, v in zip(LABELS, mean_p)},
            "windows": wins, "speech_detected": True, "latency_ms": round((time.time() - t0) * 1000, 1)}
```

The JSON answer. FastAPI converts the Python dictionary to JSON automatically.

`requirements.txt` — why each library

Library	Role
`fastapi`	the web framework
`uvicorn`	the server process that runs FastAPI (`uvicorn app:app`)
`python-multipart`	lets FastAPI parse file uploads (without it `UploadFile` fails)
`onnxruntime`	runs the neural network
`soundfile`	decodes WAV/FLAC/OGG/MP3 (needs the system library libsndfile)
`soxr`	high-quality resampling
`numpy`	arrays and maths (`<3` pins a version range the other libraries support)

`Dockerfile` — line by line

```
FROM python:3.11-slim
```

Start from a minimal Debian image with Python 3.11 (`slim` = small download, faster builds).

```
RUN apt-get update && apt-get install -y --no-install-recommends libsndfile1 curl ca-certificates && rm -rf /var/lib/apt/lists/*
```

Install the **system** library that `soundfile` needs (`libsndfile1`), `curl` (to download the model if needed) and certificates for HTTPS. `--no-install-recommends` and removing the package lists keep the image small.

```
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
```

Work inside `/app`. **Copy `requirements.txt` first and install, then copy the code**: Docker caches each step, so editing `app.py` later does not reinstall every library (faster rebuilds).

```
ARG MODEL_URL=https://github.com/ggndev/ser-project/releases/download/v1/model.onnx
RUN if [ ! -f model/model.onnx ]; then curl -fL --retry 3 -o model/model.onnx "$MODEL_URL"; fi \
   && ls -la model
```

A safety net: **if the model file is missing** from the repository, download it from a GitHub Release (web uploads are limited to 25 MB, which is why this fallback exists). When the file is in the repo, this step does nothing. `ls -la model` prints the folder into the build log so you can *see* that the model is present.

```
ENV PORT=10000 ORT_THREADS=1
CMD ["sh", "-c", "uvicorn app:app --host 0.0.0.0 --port ${PORT}"]
```

Defaults for the port and one inference thread (the free plan has a fraction of a CPU). The command starts uvicorn listening on **all network interfaces** (`0.0.0.0`) so Render can reach it;

`app:app` means "the object `app` in `app.py`". Using `sh -c` lets the shell substitute `${PORT}` — Render may provide its own `PORT` value.

`render.yaml` and `.dockerignore`

```
runtime: docker
plan: free
rootDir: backend
dockerfilePath: ./Dockerfile
healthCheckPath: /health
```

A *blueprint* describing the Render service: build with Docker, **free plan**, look inside `backend/`, and check `/health` to decide the service is healthy. (We configured the same options in Render's web form.)

`.dockerignore` lists files that Docker must not copy into the image (`__pycache__`, and the unused `model_int8.onnx`), so the image stays smaller.

Things to tell the mentor

- **Train/serve parity:** the API re-implements the notebook's preprocessing so the same audio gives the same model input.
- **Defensive design:** clear 400/422 errors, size and time limits, silence detection, a warm-up call — a free server must survive bad input.
- **Memory discipline:** one window per model call came from a real failure (an HTTP 502 that was most likely caused by running out of memory) found during testing.
- **No secrets in the API:** it is open by design; login lives in the app (see ../04-android/README.md).